

Proyecto Final

Log Anomaly Detector

Sistema híbrido de detección y clasificación de amenazas de seguridad en redes

ÍNDICE

- 01. INTRODUCCIÓN
- 02. METODOLOGÍA Y DISEÑO
- 03. IMPLEMENTACIÓN Y EVALUACIÓN
 - 3.1: PREPROCESAMIENTO DE DATOS
 - 3.2: SELECCIÓN Y ENTRENAMIENTO DEL MODELO
 - 3.3: EVALUACIÓN
- 04. CONCLUSIONES Y RECOMENDACIONES
- 05. REFERENCIAS
- 06. ANEXOS

Santiago Morgado Blonar

Máster en Inteligencia Artificial — CEI Sevilla

Introducción

Me empecé a interesar en la ciberseguridad a los pocos meses de comenzar a estudiar programación, pero fue después de unos años, trabajando en un banco como ingeniero de software, que me fui enriqueciendo de una mayor conciencia de las amenazas informáticas, especialmente en las de red, área donde estaban enfocadas las principales tareas de mi equipo. Al poco tiempo de concluir este trabajo después de más de tres años, decidí volcarme de lleno en los estudios nuevamente. Mi deseo de expandir conocimiento junto con el auge de la inteligencia artificial y la conciencia por las múltiples vulnerabilidades informáticas aprehendidas durante los años previos me llevaron a escoger este máster para continuar desarrollándome en estas áreas y, por consecuencia, este caso de estudio para el trabajo final que se trata sobre la detección de anomalías en redes y su clasificación.

El problema a resolver surge de la necesidad de reducir el volumen de alertas de seguridad que reciben los equipos SOC (Security Operations Center) constantemente, causando una demanda alta de personal capacitado (a veces, demasiado capacitado para tareas sin importancia) para intervenir en sistemas que requieren un “human in the loop” – una persona que investigue, analice, y/o clasifique la severidad de los incidentes.

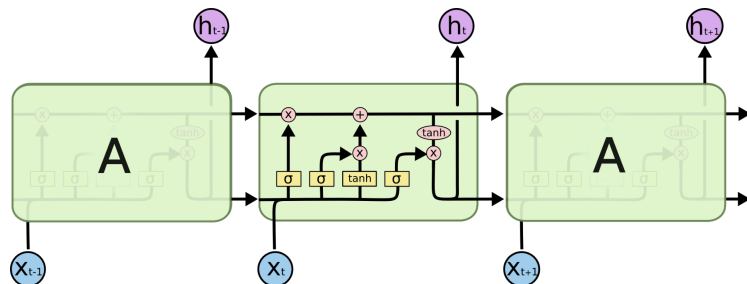
El objetivo principal de este trabajo es demostrar cómo la inteligencia artificial puede complementar las prácticas actuales de ciberseguridad mediante la detección automática de anomalías en flujos de red y la clasificación del tipo de amenaza asociada. Se usará el dataset “CIC-IDS2017” (Sharafaldin et al., 2018), referencia ampliamente utilizada en la industria por reproducir escenarios de ataque realistas en entornos empresariales.

Objetivos específicos:

- Implementar un LSTM Autoencoder entrenado exclusivamente con tráfico benigno como detector de anomalías
- Desarrollar un clasificador XGBoost multiclase que identifique el tipo de amenaza asociado a cada flujo sospechoso entrenado con tráfico malicioso. Ataques cubiertos: DoS Hulk, DoS Slowhttptest y DoS slowloris
- Integrar ambos modelos en un pipeline de dos etapas y evaluar su rendimiento conjunto mediante métricas estándar (precisión, recall, F1 y matriz de confusión)
- Identificar y documentar las limitaciones y potenciales mejoras de la arquitectura propuesta

Metodología y Diseño

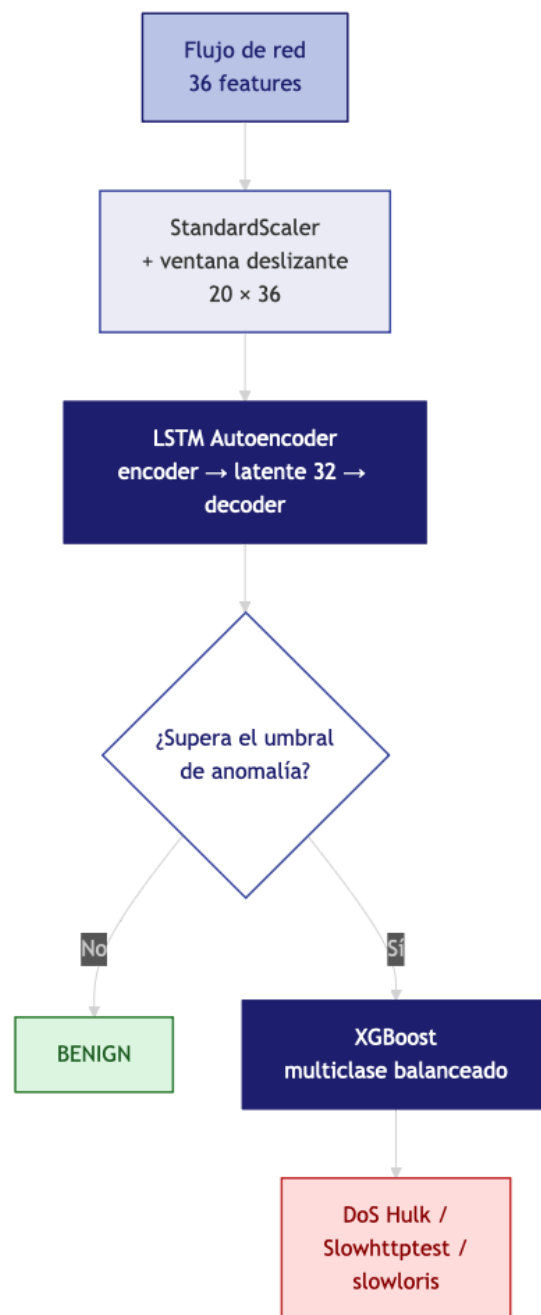
La arquitectura del proyecto está compuesta por una LSTM (Long Short Term Memory) – tipo especial de RNN (Red Neuronal Recurrente) capaz de aprender dependencias en secuencias temporales de larga duración gracias a la incorporación de tres puertas (forget, input y output) que regulan qué información se descarta, se incorpora y se propaga entre pasos de la secuencia, cuyo objetivo, integrada en una arquitectura Autoencoder, es el de aprender cómo se comportan las secuencias de flujos de red benignos para después reconstruirlas y generar un error de reconstrucción conocido. Al ser no supervisado, permite capturar anomalías de todo tipo al intentar reconstruir secuencias de flujos de red malignos cuyo error de reconstrucción es mayor al conocido.



Estructura interna de una celda LSTM. Las tres puertas (σ) regulan qué información se descarta, se incorpora al estado y se propaga al siguiente paso. Fuente: Olah (2015).

Diseño del LSTM Autoencoder:

- Encoder LSTM que transforma cada ventana temporal de 20 flujos de red consecutivos en un vector de activación
- Este vector se comprime luego mediante una capa densa a un espacio latente de 32 dimensiones



- Una vez comprimido, un RepeatVector actúa como "puente" que reconstruye la dimensión temporal/secuencial replicando el vector comprimido 20 veces (longitud de ventana escogida)
- El decoder, también LSTM, reconstruye la secuencia paso a paso
- Finalmente una capa densa de tipo TimeDistributed(Dense) proyecta cada paso temporal de nuevo a las 36 features originales

Este modelo actúa como detector de anomalías sin conocimiento previo del tipo de ataque. Sin embargo, un detector no supervisado solo informa que un flujo es anómalo, no clasifica el ataque. Para resolver esto, se incorpora un clasificador supervisado llamado XGBoost multiclase que opera sobre los flujos marcados como anómalos y determina el tipo específico de ataque (BENIGN, DoS Hulk, Slowhttptest o slowloris). Este modelo se entrenará con un subconjunto de datos seleccionado obtenido de otro día de recolección de datos del mismo sistema que contiene estas amenazas.

Diseño del XGBoost:

- Objetivo multiclase: "multi:softprob", clasificación entre BENIGN, DoS Hulk, Slowhttptest y slowloris, devolviendo probabilidades por clase
- Sample weights balanceados: se aplicó `compute_sample_weight('balanced')` para compensar la desproporción entre clases (51% BENIGN vs 30% DoS Hulk vs 11% Slowhttptest vs 8% slowloris), evitando el sesgo hacia la clase mayoritaria (BENIGN)
- Partición estratificada: se dividió el conjunto de datos en 70/30 con `stratify=y` para garantizar que ambas partes conservan la misma distribución original de clases
- Hiperparámetros principales: se evaluará inicialmente con 200 estimadores, profundidad máxima de 6 y tasa de aprendizaje de 0.1. Posteriormente se evaluará si se pueden reducir los estimadores o la profundidad para acelerar el entrenamiento en caso de ser necesario

La combinación de estos dos modelos resulta entonces en un pipeline de dos etapas: detección no supervisada seguida de clasificación supervisada, cubriendo así tanto la identificación de tráfico anómalo sin conocimiento previo como la clasificación del ataque específico.

Implementación y evaluación del modelo

Puntos a desarrollar

3.1: Preprocesamiento

3.2: Selección y entrenamiento del modelo

3.3: Evaluación

3.1: Preprocesamiento

Comencé analizando la magnitud de ambos datasets descargados correspondientes a dos días distintos (Lunes y Miércoles) provenientes de un sistema que capturó flujos de red durante una semana seguida (CIC-IDS2017 - Canadian Institute for Cybersecurity). El tamaño de estos era inmenso: +500.000 filas en el del Lunes y +600.000 filas en el del Miércoles, ambos con 79 features. Las transformaciones fueron las siguientes:

1. Limpieza de NaN e infinitos: se dropearon los NaN al ser pocos (~1000 entre ambos datasets, feature “Flow Bytes/s”) y los $\pm\text{inf}$ (valores originados por error durante la extracción de flujos usando CICFlowMeter – bug conocido y documentado)
2. Reducción del dataset con especial atención al orden (requerimiento LSTM): se seleccionaron 50.000 filas manteniendo el orden para no afectar la estructura secuencial necesaria en LSTMs. Para el dataset con anomalías, el del día Miércoles, se analizó qué rango de filas representaba de mejor manera la distribución de amenazas
3. Extracción de features relevantes para el caso de estudio: se analizó la relación entre features haciendo uso de la matriz de correlación y representando en un heatmap para destacar los pares con fuerte relación entre sí (threshold del 90%). Haciendo uso de este análisis y la documentación del dataset para identificar duplicados, copias e irrelevancias dentro del contexto, para mi desconocido, de la ciberseguridad, se logró definir una lista para eliminar las features identificadas
4. Estandarización (StandardScaler): la estandarización se aplicó únicamente sobre el tráfico benigno del lunes con el objetivo de evitar el data leakage que introduciría ajustar el escalador sobre datos que contienen los ataques a detectar
5. Adaptación de labels (LabelEncoder): los labels del dataset son cadenas de texto (BENIGN, DoS Hulk, DoS Slowhttptest, DoS slowloris). Para adaptarlas al XGBoost se aplicó LabelEncoder, que mapea cada clase a un entero. Se hizo fit_transform sobre y_train y

transform sobre los datos de evaluación. La transformación inversa (`inverse_transform`) se usó después para recuperar las etiquetas originales en el reporte final.

6. Construcción de ventanas secuenciales: algo nuevo para mí es que las RNN LSTM requieren una entrada tridimensional (filas, secuencias, features). Para adaptar el flujo de red a este formato se aplicó una ventana deslizante de 20 flujos consecutivos con stride de 1, transformando la matriz de (50.000, 36) en un tensor de (49.981, 20, 36). Este procedimiento preserva principalmente el orden secuencial, requerido al entrenar este tipo de redes neuronales. La decisión del tamaño de cada ventana (20), se llevó a cabo tras evaluar distintos valores una vez finalizado el entrenamiento. Valores mayores demoran mucho el entrenamiento sin impactar en los resultados.

3.2: Selección y entrenamiento del modelo

El LSTM Autoencoder se entrenó de forma no supervisada usando las ventanas construidas a partir del tráfico benigno del lunes (39.984 ventanas de entrenamiento y 9.997 de validación, división secuencial 80/20). Como función de pérdida se usó el error absoluto medio (MAE).

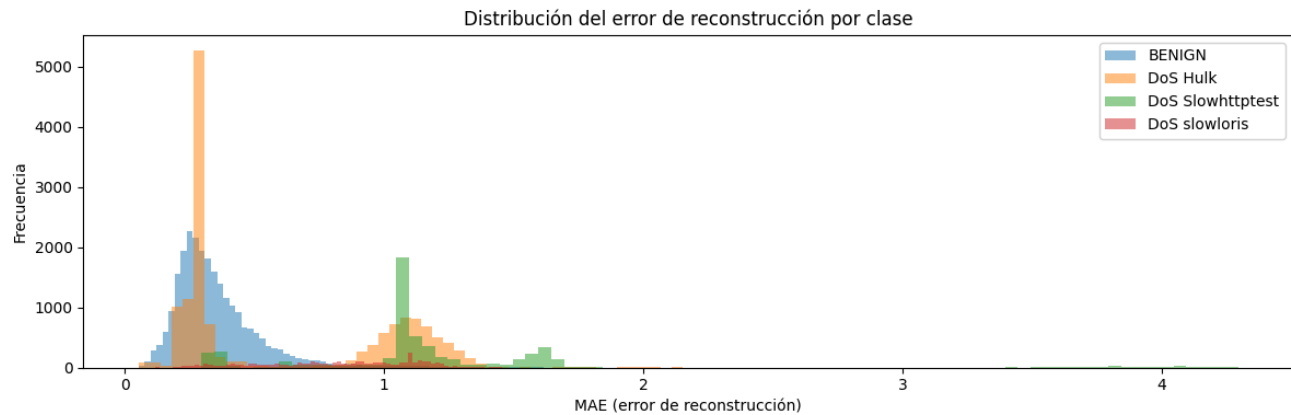
Se usó el optimizador Adam con los hiperparámetros por defecto, un tamaño de batch de 256 y un máximo de 50 épocas. Para evitar el sobreajuste se incorporó EarlyStopping monitoreando la pérdida de validación con paciencia de 5 épocas y `restore_best_weights=True` para quedarme con la mejor época. El resumen dio como output una totalidad de 55,108 parámetros entrenables.

El modelo XGBoost se entrenó de forma supervisada sobre el 70% del tráfico de miércoles (35.000 flujos con amenazas) usando la estratificación para preservar la proporción original de clases. Comencé con 200 estimadores pero post evaluación se fue iterando con menos para terminar con un total de 30, esta decisión no perjudicó casi nada los resultados y agilizo el entrenamiento. La profundidad también se fue evaluando pero 6 me pareció un valor acorde a tiempo de entrenamiento y resultado.

3.3: Evaluación

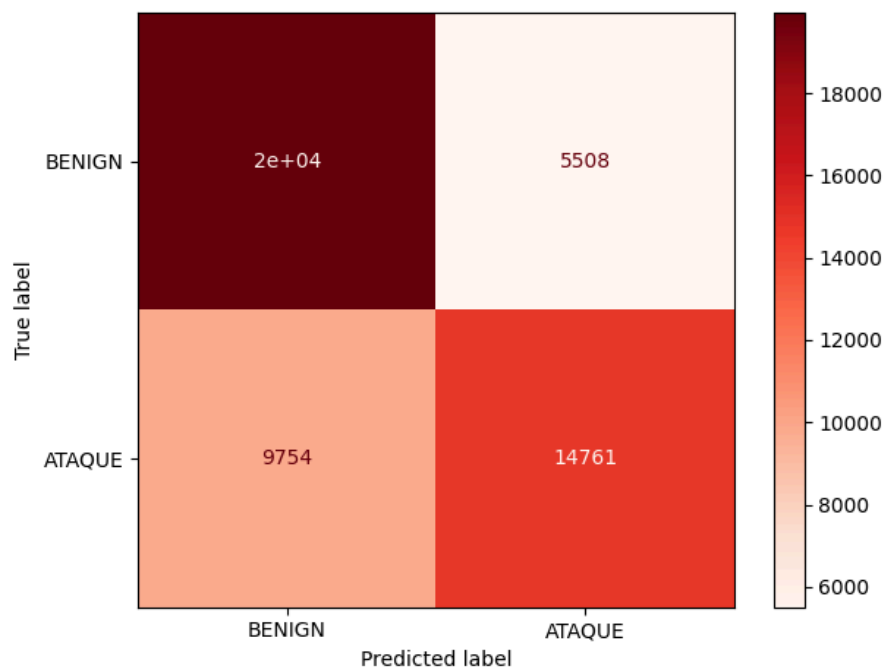
Después de terminar el fit del LSTM, convergió en ~48 épocas (variando en cada ejecución) con una pérdida de validación final de ~0.2956, sin signos de sobreajuste (se usó un gráfico para marcar la curva de entrenamiento y ambas variables de pérdida, train y val, convergieron descendiendo a la derecha). Posteriormente se evaluó sobre las 49.981 ventanas del tráfico de miércoles, que contiene tanto flujos benignos como ataques DoS reales. Cada ventana fue clasificada como BENIGN o ANOMALÍA en función de si su error de reconstrucción superaba el umbral calibrado. Este umbral se obtuvo evaluando el error de reconstrucción del conjunto de validación benigno, fijándose en el percentil 95 (valor final: 0.4426).

A continuación, dejo un gráfico que ayuda a comprender cómo se obtuvo este umbral:



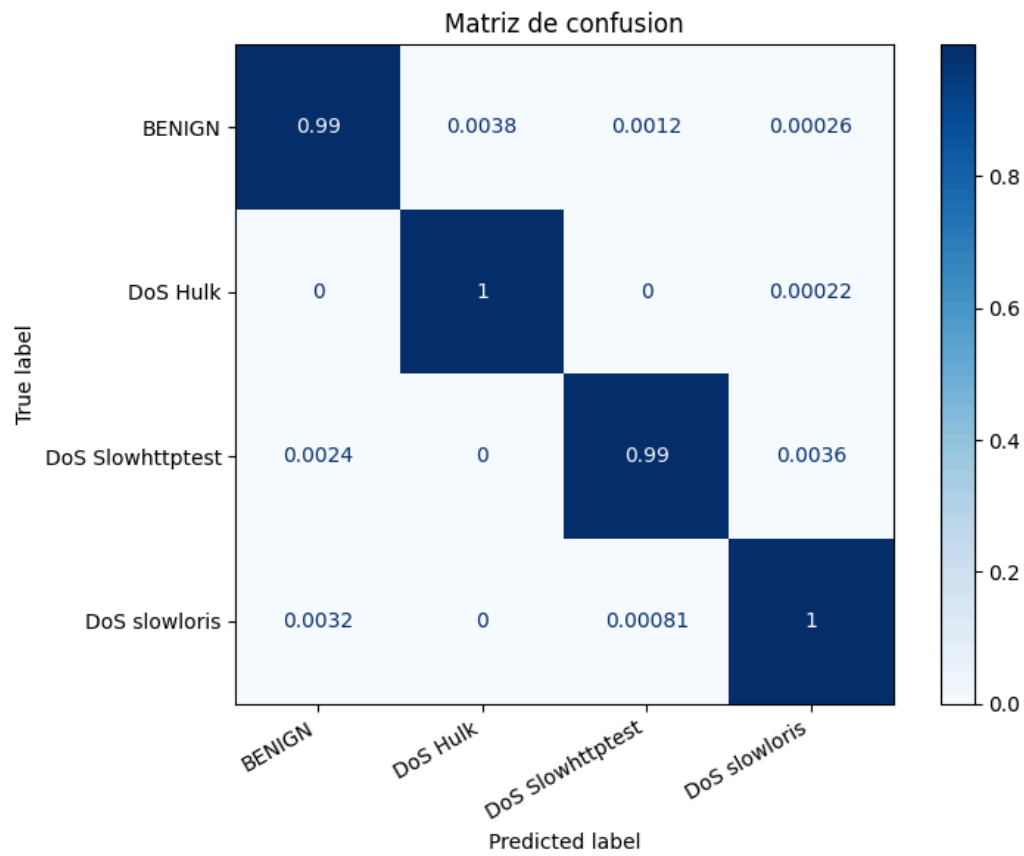
En este histograma podemos ver una separación clara de "DoS Slowhttptest" y "DoS slowloris". Ahora, también se observa como "DoS Hulk" se solapa con "BENIGN" (tráfico benigno). El motivo se puede justificar por una razón específica del tipo de ataque que "DoS Hulk" comete: genera HTTP requests aparentemente válidos que a nivel de features no son distintas al tráfico normal.

Matriz de confusión (LSTM):



XGBoost se evaluó de forma independiente sobre la partición de test (30% de Wednesday, 15.000 flujos con distribución estratificada), sin intervención del LSTM. A continuación dejo algunos gráficos de los resultados:

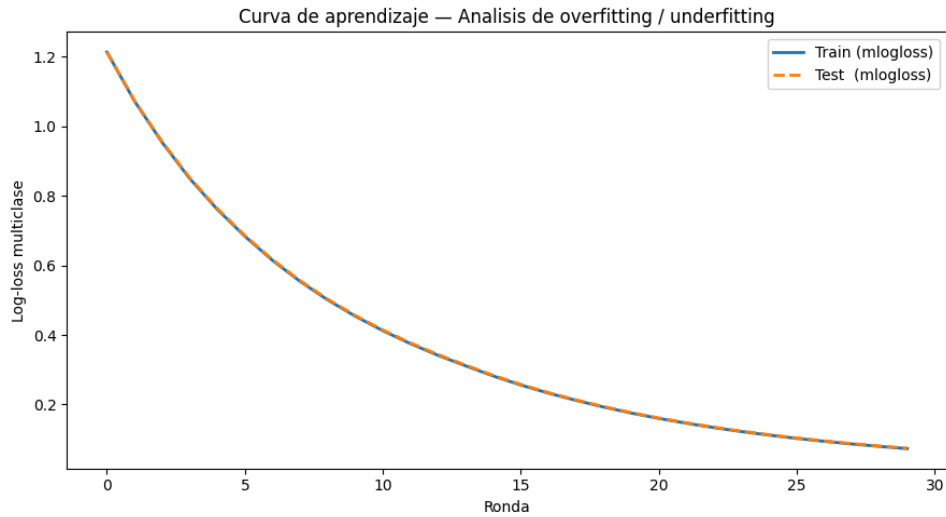
Matriz de confusión:



Reporte de clasificación:

	precision	recall	f1-score	support
BENIGN	1.00	0.99	1.00	5508
DoS Hulk	1.00	1.00	1.00	6252
DoS Slowhttptest	1.00	0.99	1.00	4967
DoS slowloris	0.99	1.00	1.00	3542
accuracy			1.00	20269
macro avg	1.00	1.00	1.00	20269
weighted avg	1.00	1.00	1.00	20269

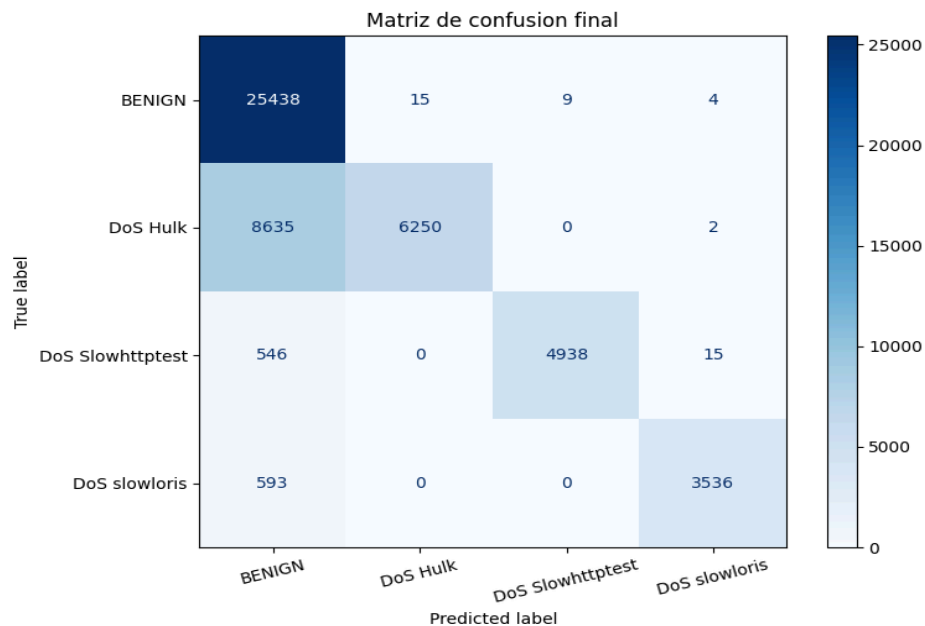
Curva de aprendizaje:



Como podemos ver, se alcanza un F1-score de 1 sobre el conjunto de test, con precisión y recall muy buenos en las cuatro clases. Este resultado confirma que las 36 features seleccionadas alcanzan para clasificar correctamente cada tipo de ataque cuando se dispone de labels durante el entrenamiento.

La evaluación completa combina ambas etapas: las ventanas marcadas como benignas por el LSTM reciben la etiqueta BENIGN directamente; las marcadas como anómalas pasan al XGBoost para su clasificación.

Matriz de confusión final (LSTM + XGBoost):



Reporte de clasificación:

	precision	recall	f1-score	support
BENIGN	0.72	1.00	0.84	25466
DoS Hulk	1.00	0.42	0.59	14887
DoS Slowhttptest	1.00	0.90	0.95	5499
DoS slowloris	0.99	0.86	0.92	4129
accuracy			0.80	49981
macro avg	0.93	0.79	0.82	49981
weighted avg	0.86	0.80	0.78	49981

Observaciones del resultado:

- Recall perfecto sobre BENIGN (1.00): el pipeline nunca clasifica tráfico legítimo como ataque, esto reduce falsas alarmas y quita tiempo de revisión manual en vano
- Precisión casi perfecta sobre los tres tipos de ataque: cuando el sistema identifica una amenaza específica, casi siempre clasifica correctamente
- F1 elevado en Slowhttptest (0.95) y slowloris (0.92): los dos ataques lentos son detectados y clasificados con alta fiabilidad
- Recall reducido en DoS Hulk (0.42): limitación heredada de la primera etapa, analizada a continuación

04

Conclusiones y recomendaciones

Conclusiones generales

Me gustaría comenzar destacando la efectividad del modelo supervisado XGBoost. Aun reduciendo los estimadores y la profundidad, el modelo se desempeñó muy bien en la clasificación de cada tipo de ataque. Una de las principales búsquedas de este trabajo era la reducción de un “human in the loop” en equipos que monitorean este tipo de amenazas. Al clasificar con tanta precisión, podemos prescindir de la revisión manual cuando surgen amenazas conocidas. Por otro lado, es muy interesante como una red neuronal recurrente puede captar secuencias anómalas cuando los tipos de ataques afectan de una manera impactante este tipo de sistemas. Al ser un modelo no supervisado, las secuencias que este determina como anómalas pueden ser conocidas como no, por lo tanto, cualquier ataque “moderno”, o desconocido se distinguiría de igual modo y pasaría a revisarse por alguien capacitado cuya función es justamente esta. Quitando la carga de revisiones innecesarias sobre ataques conocidos gracias al XGBoost, los técnicos podrían disponer del tiempo para dedicar a este tipo de análisis.

Sin embargo, una de las limitaciones de esta arquitectura comprende la incapacidad de detectar anomalías que no perturban de manera impactante las métricas del sistema. En este caso, se observó como 9.750 flujos de DoS Hulk nunca fueron evaluados por la solapación con el tráfico benigno. El error de reconstrucción no superó el umbral, por lo tanto nunca llegaron a ser clasificados por el XGBoost.

Lecciones aprendidas

Antes de llegar a esta arquitectura, el trabajo arrancó con un enfoque distinto sobre otro dataset de Kaggle. La idea inicial combinaba un LSTM Autoencoder con un Isolation Forest: cada modelo generaba su propio score de anomalía y la decisión final se tomaba fusionando ambos. Los resultados fueron muy malos, con un F1 por debajo de 0.05.

Al analizar qué estaba fallando, identifiqué que el problema no estaba en los modelos ni en los hiperparámetros, sino en los datos: estaban generados sintéticamente y no eran propios de sistemas reales. Las etiquetas no tenían relación clara con las features disponibles, así que por más ajuste que se aplicara sobre los modelos no había forma de que aprendieran algo que los datos no tenían.

A partir de ahí se tomaron dos decisiones. Primero, cambiar al dataset CIC-IDS2017 en su versión oficial (MachineLearningCVE), con features generadas por CICFlowMeter sobre capturas reales. Segundo, replantear la arquitectura hacia el pipeline actual: LSTM Autoencoder para detectar y XGBoost para clasificar.

La conclusión que queda de este proceso es que la calidad del dataset y la elección de una arquitectura adecuada al problema importan más que cualquier ajuste fino sobre los modelos, los hiperparámetros, o el procesamiento.

Recomendaciones

Una de las principales, atacar la limitación que se observó con el tipo de ataque DoS Hulk. Como se vio en la evaluación, este tipo de ataque no modifica lo suficiente las métricas del flujo como para que el LSTM lo marque como anómalo. Consideraría agregar más features que representen mejor este tipo de flujos, ampliar las ventanas deslizantes (de 20 a 100) para captar más patrones y/o combinar el modelo LSTM con otro no supervisado que detecte de una forma distinta las anomalías como el Isolation Forest.

Por otro lado, como se mencionó previamente, se redujo el dataset utilizado para garantizar un tiempo de ejecución adecuado, pero al hacerlo se dejaron varias clases de ataque fuera (PortScan, Brute Force, Web Attacks, Infiltration, Botnet). Se podría ampliar el alcance a las demás para ver primero cómo generaliza esta arquitectura. De ser positivo, el escalado permitiría potenciar los beneficios comentados previamente, dejando solo los flujos “desconocidos” para revisión.

Por último, la incorporación de una explicación al finalizar la clasificación integrando un modelo de lenguaje natural, por ejemplo:

“Se detectó una amenaza en el flujo de red x clasificado como DoS slowloris a las 15:30 PM. Razón: conexiones inusualmente largas con paquetes pequeños y espaciados, consistente con saturación lenta del servidor. Acción: aplicar rate limiting por IP origen y ajustar timeouts de conexión.”.

Cuando el pipeline detecta y clasifica un ataque, un LLM podría tomar los features con mayor error de reconstrucción, la clase que se clasificó y el contexto del flujo, y generar una explicación detallada para el personal técnico.

Referencias

Olah, C. (2015, 27 de agosto). *Understanding LSTM Networks*. colah's blog.

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Chandola, V., Banerjee, A., & Kumar, V. (2009). *Anomaly detection: A survey*. *ACM Computing Surveys*, 41(3), 1–58. <https://doi.org/10.1145/1541880.1541882>

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>

Engelen, G., Rimmer, V., & Joosen, W. (2021). Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. *2021 IEEE Security and Privacy Workshops (SPW)*, 7–12. <https://doi.org/10.1109/SPW53761.2021.00009>

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. (2016). LSTM-based encoder-decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148*. <https://arxiv.org/abs/1607.00148>

Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 108–116. <https://doi.org/10.5220/0006639801080116>

Canadian Institute for Cybersecurity. (2018). CIC-IDS2017 dataset (MachineLearningCVE). University of New Brunswick. <https://www.unb.ca/cic/datasets/ids-2017.html>

XGBoost Developers. (2022). XGBoost documentation (v1.7). <https://xgboost.readthedocs.io>

06

Anexos

Uso de IA

Durante el desarrollo de este trabajo se utilizaron herramientas de IA generativa (Claude de Anthropic y ChatGPT de OpenAI) como apoyo en tareas puntuales: depuración de errores en el código, consulta sobre APIs de Keras/XGBoost, redacción de comentarios técnicos y corrección de redacción en la memoria. Todo el código generado con apoyo de IA fue revisado, validado y adaptado manualmente antes de incorporarse al proyecto.

A continuación se incluyen, a modo de ejemplo, algunos de los prompts utilizados durante el desarrollo del trabajo:

1) *"I'm working on the feature engineering step. Trying to remove some columns. There are a lot of correlated variables (just 67 over the 0.9 correlation threshold). What'd you suggest?"*

2) *"I'll move on with the data division and training. I'm thinking of training the XGBoost first, but first give me an overview of the model: pros, cons, and how it works. I was thinking of using an LSTM Autoencoder to detect anomalies + XGBoost for classification."*

3) *"Let's define the steps for the LSTM Autoencoder model. Ensure to justify (in spanish) each decision (I've never worked with LSTM RNNs) and leave references. Guide me through each step, and wait for me to understand it and test it before moving to the next one. Before starting, make sure to read MY subset of the CIC-IDS2017 (50k rows) to make sure that the LSTM architecture is aligned and makes sense. Consider the following article I've read to explain to me how it works: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>."*

4) *"I'm attaching the current draft of my final project document (.docx). Using the CEI evaluation criteria PDF I shared earlier, go through it section by section and tell me what you think. I want to keep my own tone, so don't rewrite anything — just flag inconsistencies, errors, or things that don't add up. When you're done, fix the cover page (it's misaligned). Return the file in the same format so I can keep editing it in Google Docs."*

Notebook

El código completo del pipeline (preprocesamiento, entrenamiento LSTM, entrenamiento XGBoost, evaluación y visualizaciones) se encuentra en el notebook `santiago_morgado_ia_final_project.ipynb` entregado junto con esta memoria.

Dataset

CIC-IDS2017 — MachineLearningCVE (Canadian Institute for Cybersecurity, Universidad de New Brunswick). Archivos utilizados: Monday-WorkingHours.pcap_ISCX.csv (tráfico benigno, entrenamiento del LSTM) y Wednesday-workingHours.pcap_ISCX.csv (tráfico mixto con ataques DoS Hulk, Slowhttptest y slowloris, entrenamiento y evaluación del XGBoost).

Entorno de ejecución

Google Colab (GPU T4), Python 3.11. Librerías principales: TensorFlow/Keras 2.15, XGBoost 2.0, scikit-learn 1.3, pandas 2.1, numpy 1.26, matplotlib 3.8 y seaborn 0.13.

Material complementario

Se incluyen como material complementario los gráficos de evaluación (histograma de errores de reconstrucción, matrices de confusión LSTM / XGBoost / pipeline, curva de aprendizaje y reportes de clasificación) integrados en la sección 03 de esta memoria.